实验六 K-Means 聚类实验

姓名　顾凯琪

学号　226001624

学院 信息学院

专业班级 计算机 2 班

一、 实验目的

1. 掌握聚类的基本原理与使用场景；
2. 使用给定的数据集进行 K-Means 聚类操作，并进行可视化操作，以增进对 K-Means 的原理理解；
3. 学习常用的几种聚类方法的实现原理与特点，比较其与 K-Means 之间的差异与优缺点；
4. 熟悉 Python 集成开发环境的使用方法。

二、 实验环境

操作系统：

Windows 11

集成开发环境：

1. Anaconda3
2. PyCharm 2022.2.2

相关 Python 库：

1. python3.9
2. numpy
3. matplotlib（3.8.3）
4. sklearn（scikit-learn1.4.1）

三、 实验内容

1. 使用 Python 实现在给定的数据集上进行 K-Means 聚类操作；

2. 对数据以及聚类结果进行可视化绘图操作；

3. 在实验过程中使用不同的簇的个数与不同的初始化簇心的方法。

四、 实验步骤：

1. 数据集加载

实验指导中给出了一个应用文件 utilities.py，其中包含数据加载、模型评价等常用的方法，使用数据加载方法从实验数据文件 data_multivar.txt 中进行数据集的读取。

2. 可视化绘图

实验给定的数据样本各含有两个特征，可以考虑使用二维散点图的方式进行数据的绘制，以便后续的聚类结果的分析与验证。使用 matplotlib.pyplot 库中的 scatter 函数即可实现散点图的绘制，同时可以进行样式的调整，如点的大小、轮廓等。

3. 定义 K-Means 模型并训练

使用 sklearn 机器学习库中集成的的 KMeans 模型进行聚类操作，相关参数包括初始化方法 init、聚类数 n_clusters、随机初始化的次数 n_init。首先将初始化方法设置为 k-means++，即为随机选定一个簇心，随后计算所有点到该簇心的距离，距离越大则后续被选为簇心的概率越大，重复上述过程直到确定所有簇心。

4. 对聚类结果进行绘图

聚类完成后将确定各个簇心，在绘制散点图时将簇心特别标注，此外对空间进行划分，即精确到空间的任意位置属于哪一类。具体的绘制方法详见代码内容。

5. 调整簇的数目与簇心初始化方法

尝试调整一些参数，例如簇心的初始化方法、迭代次数、簇的数目等，分析调整参数后的聚类结果，总结各个参数对聚类结果的影响。并使用轮廓系数法确定最优的聚类簇数。

五、 代码提交

该部分将按照代码逻辑与以上实验步骤进行代码的划分，包括数据的获取与绘制、K-Means 的模型训练与结果绘图。

```
"""
------- 导入涉及到的头文件与库函数-----
"""
```

```
import numpy as np
import matplotlib

matplotlib.use('TkAgg')
import matplotlib.pyplot as plt
from sklearn import metrics
from sklearn.cluster import KMeans
# 实验指导给定的程序包
import utilities
```

```
"""
-------导入数据与绘图-----
"""
```

```
def get_data():
    data, _ = utilities.load_data('data_multivar.txt')
    print(data)
    return data

def plot_data(data):
    x = data[:, 0]
    y = data[:, 1]
    plt.figure()
    plt.scatter(x,y,marker='o',facecolors='none',edgecolors='k',s=30)
    x_min, x_max = min(x) - 1, max(x) + 1
    y_min, y_max = min(y) - 1, max(y) + 1

    plt.xlim(x_min, x_max)
    plt.ylim(y_min, y_max)
    plt.xticks(())
    plt.yticks(())
    plt.title('Input Data')
    plt.show()
```

```
"""
-------模型的训练与结果绘图-----
"""
def kmean_train(data):
    # 初始化方法、簇的数目、初始化随机次数
    init_method = 'k-means++'
    num_clusters = 4
    init_random_num = 10
    # 定义 K-Means 模型
    kmeans = KMeans(init=init_method, n_clusters=num_clusters, n_init=
init_random_num)
    # 数据拟合
    kmeans.fit(data)
    # 绘图
    plt.figure()
    plt.clf()
    x = data[:, 0]
    y = data[:, 1]
    # 绘制原始数据图像
    plt.scatter(x,y,marker='o',facecolors='none',edgecolors='k',s=30)
    # 获取簇心数据
    center = kmeans.cluster_centers_
    # 对于簇心进行绘图
    plt.scatter(center[:, 0], center[:, 1], marker='o', s=100, linewid
ths=3, color='k', zorder=10, facecolors='red')
    # 数据范围
    x_min, x_max = min(x) - 1, max(x) + 1
    y_min, y_max = min(y) - 1, max(y) + 1
    # 调整绘图的数据范围
    plt.xlim(x_min, x_max)
    plt.ylim(y_min, y_max)
    # 去除坐标刻度
    plt.xticks(())
    plt.yticks(())
    plt.title('KMeans init:{}, num_clusters:{}, n_init:{}'.format(init
_method, num_clusters, init_random_num))
    # 网格粒度
    step_size = 0.01
    # 获取一个网格，根据数据范围进行生成
    x_values, y_values = np.meshgrid(np.arange(x_min, x_max, step_size
), np.arange(y_min, y_max, step_size))
    # 对于网格上的每一个点传入训练好的 K-Means 进行分类
    predicted_labels = kmeans.predict(np.c_[x_values.ravel(), y_values
.ravel()])
```

```
    # 将分类结果进行形状调整
    predicted_labels = predicted_labels.reshape(x_values.shape)
    # 绘图，与原数据图像一起
    plt.imshow(predicted_labels, interpolation='nearest',
               extent=(x_values.min(), x_values.max(), y_values.min(),
y_values.max()),
               cmap=plt.cm.Paired,
               aspect='auto', origin='lower')
    plt.show()
```

```
"""
-------使用轮廓系数法确定最优簇数-----
"""
```

```
def find_best_k(data):
    init_method = 'random'
    init_random_num = 10
    k_list = []
    scores = []
    # 定义K-Means 模型
    for k in range(2, 10):
        # 聚类簇数为k
        kmeans = KMeans(init=init_method,
                        n_clusters=k, n_init=init_random_num)
        # 拟合数据
        kmeans.fit(data)
        # 获取标签
        labels = kmeans.labels_
        # 计算轮廓系数
        scores.append(silhouette_score(data, labels))
        k_list.append(k)
    plt.plot(k_list, scores)
    plt.show()
```

六、 实验结果与分析

1. 数据的导入

导入数据，数据的样本数为 300，每个样本含有两个特征，以该特征为横纵坐标值进行绘图操作，结果如图 1 所示。可以看到部分数据具有高度重叠的特性，且呈现明显的区分度。

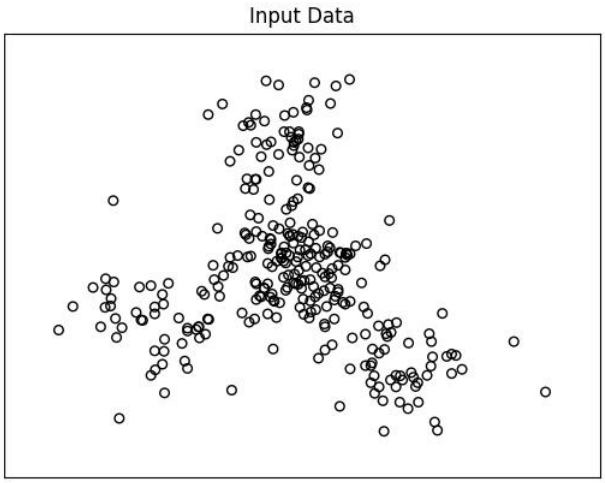


图 1. 数据内容与绘图

2. 分类结果

训练 K-Means 模型，定义初始化方法为“k-means++”，即随机选定一个簇心后根据样本与簇心的距离决定后续的选定方法，设定分类的簇数为 4，结果如下图 2 所示。

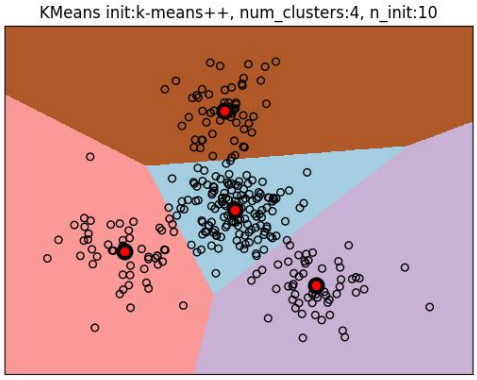


图 2. 分类结果（k-means++方法初始化、4 类）

3. 参数调整

尝试调整聚类模型的相关参数，首先进行初始化方法的调整，调整为“random”方法，其余参数保持不变，聚类结果如图 3 所示。

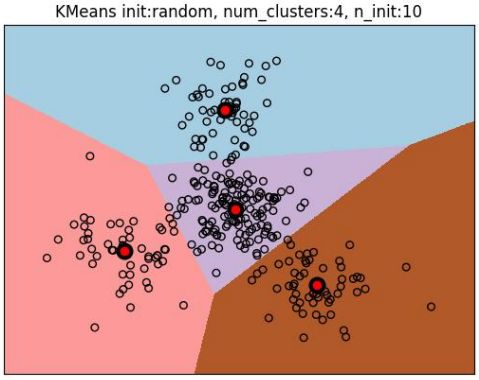


图 3. 分类结果（random 方法初始化、4 类）

可以发现，聚类的结果与 k-means++方法相同，经过多次尝试结果均相同，虽然 random 方法可能会导致最终收敛的簇心不同，但在本数据上，由于数据的分类程度较为明显，因此最终收敛的状态较唯一。

调整聚类簇数为 6，设置初始化方法为 random，并多次尝试数据的拟合，结果大致相同，如图 4 所示。

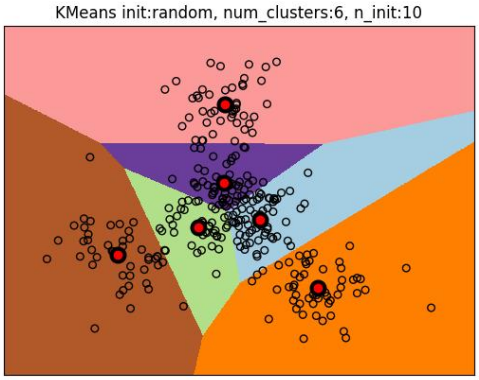


图 4. 分类结果（random 方法初始化、4 类）

从结果可以看到，数据空间被划分为了 6 类，符合实验预期。

4. 最优簇数的确定

聚类作为无监督学习方法，具体的簇数人为给出，而具体哪一个簇数是合适的，可以使用轮廓系数来确定最优簇数。轮廓系数用于描述聚类结果的簇内相似度和簇间不相似度，合理的聚类结果应该保证簇间不相似度较大且簇内相似度较小。可以使用 silhouette_score()函数实现。

选定不同的簇的个数进行聚类操作，同时记录每一种分类结果的轮廓系数，并进行绘图，如图 5 所示。

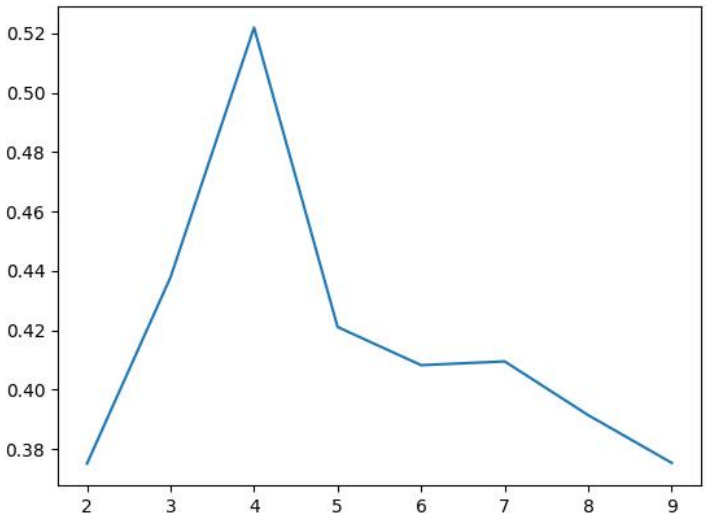


图 5. 轮廓系数值确定最优簇数

根据轮廓系数可以看到，最优簇数应选定为 4，对应的结果即如图 2 所示，符合最初的实验预期。

七、 讨论、心得

本次实验基于 Python 的 sklearn 机器学习库实现了使用 K-Means 方法进行聚类，并在给定的数据集上进行了无监督分类，得到了合理的结果。

对于 k-means 聚类方法，通过反复迭代更新分类结果以确定新的簇心，最终收敛得到一个分类结果，实现对样本空间的合理划分。本次实验通过可视化方法对聚类结果进行呈现，以加深对于聚类方法与结果的理解，同时也通过对相关参数的调整，例如簇心的初始化方法，发现在数据呈现明显的分类规律时，最终的簇心总能够得到唯一确定的结果，但如果分类结果不那么明显，最后的簇心收敛结果会随着随机的初始簇心的变化而呈现一定的变化。

此外，聚类作为一种常用的无监督学习方法，无需标签参与模型的训练，但对于簇数，仍需要训练人员进行确定，因此可以考虑使用轮廓系数来描述某一分类结果的合理性，并确定最优簇数为具有最大轮廓系数的分类器对应的簇数，本次实验经过一定的搜索与绘图，确定了最优簇数为 4，这符合实验之前对于数据

的直观分析结果。